

1-Task Title: Checking Equivalence between $p \rightarrow (q \& \& r)$, $(p \rightarrow q) \& \& r$ propositional logic

Code:

```
from prettytable import PrettyTable

table=PrettyTable()
table.field_names=["p","q","r","q ∧ r","p->q","p->(q ∧ r)","(p->q) ∧ r"]

check="is "
for p in range(0,2):
    for q in range(0,2):
        for r in range(0,2):
            table.add_row([
                bool(p),
                bool(q),
                bool(r),
                bool(q and r),
                bool(not p or q),
                bool(not p or (q and r)),
                bool((not p or q) and r)
            ])
            a=not p or (q and r)
            b=(not p or q) and r
            if a!=b:
                check="is not ";

print(table)
print("From Truth Table it can be said that  $p \rightarrow (q \wedge r)$  and  $(p \rightarrow q) \wedge r$  "+check+"EQUIVALENT\n")
```

Output:

p	q	r	$q \wedge r$	$p \rightarrow q$	$p \rightarrow (q \wedge r)$	$(p \rightarrow q) \wedge r$
False	False	False	False	True	True	False
False	False	True	False	True	True	True
False	True	False	False	True	True	False
False	True	True	True	True	True	True
True	False	False	False	False	False	False
True	False	True	False	False	False	False
True	True	False	False	True	False	False
True	True	True	True	True	True	True

From Truth Table it can be said that $p \rightarrow (q \wedge r)$ and $(p \rightarrow q) \wedge r$ is not EQUIVALENT

Press any key to continue . . . |

Discussion:

Equivalence means if 2 or more propositional logic's output is same for all the value of combination. Here $p \rightarrow (q \& \& r)$, $(p \rightarrow q) \& \& r$ are different propositional logic and there are 3 variables in the logical statement. As there are 3 variables, so total value of combination will be 2^3 or 8. But these 2 propositional statement are not equivalence because when p, q, r are false and p, r false & q is true, the outputs are not same. So $p \rightarrow (q \& \& r)$, $(p \rightarrow q) \& \& r$ propositional logic are not equivalence.

2-Task Title: Nested Quantifier 1. $\forall_x \exists_y P(x, y), P(x, y): x+y>0$
2. $\exists_x \forall_y P(x, y), P(x, y): x-y<0$

Code-1:

```
x=[]
y=[]

a=int(input("Total Number of value of x->"))
b=int(input("Total Number of value of y->"))

print("taking value of x")
x=list(map(int,input().split()))

print("taking value of y")
y=list(map(int,input().split()))

b=True
for i in x:
    a=False
    for j in y:
        m=bool(i+j>0)
        a=m or a
    if not a:
        b=False
        break

print(f"\nfor  $\forall x \exists y P(x,y)$  ,  $P(x,y): x+y>0$  statement is {b}")
```

Output:

```
Total Number of value of x->3
Total Number of value of y->4
taking value of x
7 4 3
taking value of y
-8 -6 -5 -9

for  $\forall x \exists y P(x,y)$  ,  $P(x,y): x+y>0$  statement is False
Press any key to continue . . . |
```

```
Total Number of value of x->3
Total Number of value of y->4
taking value of x
7 4 3
taking value of y
-8 -6 -2 -9

for  $\forall x \exists y P(x,y)$  ,  $P(x,y): x+y>0$  statement is True
Press any key to continue . . . |
```

Code-2:

```
x=[]
y=[]

a=int(input("Total Number of value of x->"))
b=int(input("Total Number of value of y->"))

print("taking value of x")
x=list(map(int,input().split()))

print("taking value of y")
y=list(map(int,input().split()))

a=True
for i in x:
    a=True
    for j in y:
        m=bool(i-j<0)
        a= (a and m)
    if a:
        break

print(f"\nfor  $\exists x \forall y P(x,y)$  ,  $P(x,y): x-y<0$  statement is {a}")
```

Output:

```
Total Number of value of x->3
Total Number of value of y->4
taking value of x
7 4 3
taking value of y
8 6 2 9

for  $\exists x \forall y P(x,y)$  ,  $P(x,y): x-y<0$  statement is False
Press any key to continue . . . |
```

```
Total Number of value of x->3
Total Number of value of y->4
taking value of x
7 4 3
taking value of y
8 6 5 9

for  $\exists x \forall y P(x,y)$  ,  $P(x,y): x-y<0$  statement is True
Press any key to continue . . . |
```

Discussion:

From this task I can learn about the quantifiers and also the nested quantifiers. Quantifiers is a symbol/operator used to indicate the scope of a variable and express the extent to which a statement is True. 2 types of quantifiers I have learned and these are Universal Quantifier and Existential Quantifier. First expression $\forall x \exists y$ means for all value of x, for some value of y -the expression is True or False. Second expression $\exists x \forall y$ means for some value of x, for all value of y- the expression is True or False.